Chunking and Document Segmentation

pig7selene

September 5, 2026

Abstract

Retrieval-Augmented Generation rarely searches an original source document as one indivisible object. Instead, an ingestion pipeline segments documents into retrievable units, attaches representations and metadata to those units, and indexes them for later selection. This chapter explains why the segmentation policy determines the evidence a retriever can expose, compares length-, boundary-, structure-, and semantics-based policies, and develops the trade-offs in size, overlap, hierarchy, provenance, and maintenance. It closes with operational contracts that make chunk boundaries reproducible, debuggable, and safe to evolve.

1 Introduction

Chapter 26 introduced the offline path of a Retrieval-Augmented Generation (RAG) system: source documents are parsed, represented, and indexed before a user query arrives. The apparently small step between parsing and representation is consequential. A retriever usually does not search a whole manual, paper, repository, or book as a single item. It searches units created by a **segmentation** or **chunking** policy:

$$\text{source document} \rightarrow \text{segmentation} \; \text{retrieval units} \rightarrow \text{representations} \; \text{index}. \tag{1}$$

The boundaries selected by that policy determine what can later be retrieved together. If an assumption is separated from the theorem it qualifies, a query may retrieve a statement without its scope. If unrelated sections are placed in one large unit, a relevant match can consume context with material that weakens the answer. Chunking is therefore not a cosmetic preprocessing step. It defines the interface between source structure, the retriever described in Chapters 27 and 28, and the context constructor described in Chapter 26.

This chapter treats chunking as a retrieval-system design problem rather than a universal algorithm. It does not derive BM25, dense similarity, approximate nearest-neighbor search, reranking, query rewriting, or a full RAG evaluation protocol. Those components operate on the units defined here. The same segmentation policy can behave differently with another embedding model, query distribution, or context budget, so a useful policy is a measured system choice rather than a fixed percentage copied between corpora.

2 Retrieval Units and Segmentation Boundaries

A **source document** is the original object supplied to ingestion: for example, a policy page, technical paper, source file, manual, or database record. A document can contain nested **sections** and **subsections**, and each section can contain one or more **passages**. A **chunk** is a concrete span emitted by a segmentation policy. A **retrieval unit** is the object indexed and returned by a retriever. In a simple pipeline, one chunk is one retrieval unit; in a hierarchical system, a small child chunk may be indexed while a larger parent passage is later supplied to the generator.

Let a parsed document be represented as a token sequence $x = (x_1, \ldots, x_T)$ under the tokenizer contract from Chapter 1. A segmentation policy g produces an ordered collection

$$g(x) = (c_1, \ldots, c_M), \quad c_j = x_{a_j}, \ldots, x_{b_j}, \quad 1 \le a_j \le b_j \le T. \tag{2}$$

The expression in Equation 2 does not require chunks to be disjoint, equal in length, or even determined only from token positions. A structure-aware policy can inspect headings and code fences; a semantic policy can inspect changes in meaning. What every policy must decide is which source span and surrounding information each c_j represents. Dense Passage Retrieval, for example, treats passages as the basic indexed objects and uses a fixed-length passage construction in its open-domain question-answering setting [1]. That choice is one useful baseline, not a definition of a correct RAG chunk.

3 Length-Based and Linguistic Segmentation

The simplest family cuts a stream after a fixed number of characters, words, or tokens. Character-based chunking is easy to apply when a parser supplies only plain text, but a character count is weakly related to the model's actual context consumption. Word-based rules are more readable to humans, yet their relation to downstream token count varies by language and tokenizer. Token-based chunking makes the budget explicit: its length is measured in the same units that compete for the generator's context window.

For a token window of width w and stride s, where $0 < s \leq w$, a sliding-window policy can emit

$$c_j = \left(x_{1+(j-1)s}, \ldots, x_{\min(T,(j-1)s+w)}\right). \tag{3}$$

Adjacent windows overlap by $w - s$ tokens whenever $s < w$. This construction is deterministic, simple to parallelize, and easy to re-run after a source revision. It also makes index cardinality predictable. Its blind spot is meaning: it can split a sentence, detach a heading from its content, or join the final paragraph of one topic to the first paragraph of another.

Sentence- and paragraph-based policies start from natural linguistic boundaries. Respecting these boundaries usually produces units that are easier to read and attribute, and it reduces arbitrary cuts inside an argument. Paragraphs alone, however, can be highly uneven: a one-line list item and a multi-page technical section are both paragraphs in some source formats. A practical policy can therefore accumulate whole sentences or paragraphs toward a token budget, then begin a new chunk at the nearest acceptable boundary. This is still a length-aware policy; the linguistic boundary is a constraint on where it may cut.

Policy family	Useful property	Characteristic risk
Fixed character, word, or token windows	Deterministic size and simple operational behavior	Can cut through syntax, sentences, or a local argument
Sentence or paragraph accumulation	Preserves readable local boundaries	Uneven units and no guarantee that adjacent paragraphs share one topic
Structure-aware segmentation	Retains declared document hierarchy and typed regions	Depends on reliable parsing and source markup
Semantic segmentation	May place boundaries near topic changes	Adds model cost, thresholds, and possible unstable boundaries

Table 1: Segmentation policies expose different assumptions. A robust corpus may use more than one policy for different source types, but each index should record the policy that created its units.

4 Structure, Semantics, and Context Preservation

Structure-aware chunking uses boundaries already declared by the source. Markdown and HTML expose headings, lists, tables, and code fences; well-formed documents may expose sections and subsections; source code exposes modules, classes, and functions. A conceptual policy first identifies this hierarchy, recursively segments an oversized structural unit, and attaches the hierarchy to every resulting chunk:

$$\text{document hierarchy} \longrightarrow \text{structural boundaries} \longrightarrow \text{oversized sections}$$
$$\longrightarrow \text{recursive segmentation} \longrightarrow \text{chunks plus hierarchy metadata}$$

The point is not that headings are always semantically correct. A heading can be vague, and PDF extraction can lose the reading order that made a table or caption intelligible. The point is that source structure often supplies evidence about which text should remain together. Structure-aware policies should degrade explicitly when the parser cannot provide trustworthy boundaries, rather than silently claiming the same fidelity for clean HTML and a poorly extracted scan.

Semantic chunking seeks boundaries at changes in content rather than only in length or markup. A system might compare adjacent sentence embeddings, inspect discourse cues, or use a model to propose a topical split. This can avoid grouping two unrelated topics merely because they occupy the same paragraph or window. It can also be expensive at ingestion time, dependent on the chosen model and threshold, and unstable when small source edits alter later decisions. Semantic similarity is not a guarantee that a boundary preserves all logical dependencies: a definition and its exception may look lexically or geometrically different while still needing to be retrieved together.

Context preservation is the common goal. It asks whether a unit carries enough of the surrounding material to support its likely use. An answer may require a definition and its condition, a heading and the text it scopes, a question and its answer, or a function signature and its implementation. Overlap and structure-aware chunking address this problem differently. Overlap duplicates a neighborhood around every boundary; structure-aware segmentation attempts to place fewer boundaries through meaningful units. Neither removes the need to inspect what the generator actually receives.

5 Chunk Size, Overlap, and Boundary Effects

Chunk size is a three-way allocation problem among retrieval specificity, context completeness, and system cost. Smaller chunks often isolate a claim or a definition, reducing irrelevant material in a retrieved result. They also create more vectors or postings, increase index cardinality, and make it easier to separate evidence that must be read together. Larger chunks preserve surrounding context and reduce the number of units, but they can mix several concepts, lower ranking specificity, and consume more of the context budget from Chapter 26.

No numerical size is optimal independently of the corpus. The appropriate range depends on source structure, query granularity, tokenizer behavior, embedding model, retrieval method, context length, and the downstream task. A factual lookup can favor a tightly localized unit; a legal interpretation or a multi-step technical explanation may require a larger parent section. The right question is not whether a chunk is generally “small” or “large,” but whether the retrieved unit carries the evidence required for the intended answer without crowding out stronger evidence.

Overlap offers a direct response to boundary loss. Under Equation 3, a fact near the end of one window also appears near the beginning of the next. This improves the chance that a query whose evidence crosses a boundary retrieves at least one adequate unit. The same duplication increases embedding work, storage, index entries, and the likelihood that top-k results contain near-identical text. A context constructor must consequently detect redundant spans rather than treating each retrieved identifier as independent evidence.

The distinction matters operationally. Excessive overlap can make apparent retrieval recall look better because the same source span is represented many times, while adding little new evidence to the generated context. Conversely, zero overlap can create brittle failures at a single arbitrary cut. Measure the effect using the task’s candidate recall, context redundancy, answer quality, and latency rather than assuming a universal overlap ratio.

6 Metadata, Hierarchy, and Parent–Child Retrieval

Every retrieval unit should preserve a reversible link to its source. At a minimum, a record should carry a stable document identifier, a source revision, a chunk identifier, an ordered source span, its text representation, and any policy-relevant metadata. Useful additional fields include a document title, section path, page or anchor location, timestamp, author, language, content type, and access-control

attributes. Chapter 26 explains why these fields are necessary for filtering, provenance, citation, freshness, and debugging; segmentation is the point at which many of them can otherwise be lost.

A stable chunk identifier must be designed carefully. An identifier derived only from ordinal position changes after an insertion near the document's beginning. An identifier derived only from text can collide across repeated boilerplate and changes when harmless normalization changes. In practice, the identity contract should name the source revision, span rule, segmentation-policy version, and canonicalization procedure. It must be possible to distinguish a new representation of the same source span from a genuinely different source span.

Parent–child retrieval separates the unit used for matching from the unit used for generation. Small child chunks are indexed for specificity. When one is selected, the system maps it to a parent section or a bounded local expansion before context construction:

$$\text{small child chunk} \longrightarrow \text{precise retrieval} \longrightarrow \text{parent section or local neighborhood} \longrightarrow \text{context construction} \longrightarrow \text{richer evidence}$$

This pattern can reduce the tension between precise matching and complete context, but it changes the interface: evaluation must distinguish whether the child was retrieved, whether the chosen parent contains the necessary evidence, and whether expansion overflowed the context budget. Hierarchical approaches such as RAPTOR make the broader idea explicit by organizing text at multiple levels of abstraction for retrieval over long documents [2]. Parent–child retrieval does not require learned summaries or a tree index; even a deterministic section hierarchy is a useful form of parent relation.

7 Special Document Forms and Long Contexts

Tables are not ordinary prose with decorative whitespace. A cell value can be uninterpretable without its row label, column header, unit, caption, and nearby qualifier. Naively flattening a table into independent token windows can destroy those relations. A table-aware pipeline should retain headers, row and column associations, caption or source identity, and a declared serialization rule. A system may index a whole small table, coherent row groups with repeated headers, or a text representation paired with the original structured object; the appropriate choice depends on the questions the corpus is expected to answer.

Code has analogous structural dependencies. Arbitrary windows can separate a function from its signature, a method from the class defining its fields, or a call site from imported names. A code-aware policy can prefer module, class, function, method, and block boundaries, while retaining file path, language, symbol name, imports, and line ranges as metadata. It need not make every function a single chunk: very large functions may still require an explicit subdivision rule. The requirement is to preserve the syntax and provenance that give retrieved text its meaning.

Long documents amplify every segmentation decision. Books, standards, technical manuals, papers, and legal texts contain hierarchy, cross-references, and concepts revisited at several granularities. Treating such a document as one flat sequence either produces enormous units or loses the hierarchy that helps a system reconstruct local context. A long-document pipeline should preserve the section path and ordering even if its first-stage index is flat. This keeps later expansion, citation, filtering, and reassembly possible without re-parsing an opaque chunk payload.

8 Re-Chunking, Evaluation, and Failure Modes

Changing a chunking policy changes the retrieval corpus. New boundaries can alter chunk identifiers, document-to-chunk mappings, representations, embeddings, sparse features, index entries, and evaluation judgments. A production policy change should therefore be treated as a versioned re-indexing event, not as a harmless configuration edit. Incremental changes are safe only when the system can prove which source revisions and derived units remain compatible with the published index.

Chunking should be evaluated through the downstream system, not aesthetic inspection alone. Relevant signals include retrieval recall and precision, whether an answer-supporting span reaches

the context, answer quality, duplicate-context rate, index size, ingestion cost, and query latency. The diagnosis must remain decomposed. Poor retrieval may arise from a weak embedding model or ANN index as Chapters 27 and 28 explain, but it may also arise because the evidence was separated, stripped of metadata, or never emitted as a suitable unit.

Common failures follow directly from this interface. Units that are too small lose conditions and surrounding definitions; units that are too large blur multiple topics and waste context. Arbitrary cuts can break sentences, tables, and code. Excessive overlap produces duplicate results and inflates storage; missing overlap can make one cut decisive. Lost headings or source metadata damage provenance. Semantic policies can create implausible boundaries, and non-deterministic policies make re-indexing difficult to audit. The response is not to select the most sophisticated policy by default, but to make the policy observable, versioned, and evaluated against the target workload.

9 Implementation Contracts

The **input contract** must declare accepted source formats, parser versions, canonicalization rules, tokenizer revision where token budgets matter, and the handling of malformed or unparseable content. The **segmentation contract** must name the policy family, target and maximum unit lengths, permissible boundaries, stride or overlap rule, structure-recognition rules, treatment of tables and code, and deterministic tie behavior. It should make clear whether a chunk is a source span, a rendered representation, or both.

The **provenance contract** must preserve document and chunk identifiers, source revision, ordered span offsets, section path, timestamps, policy metadata, and access-control fields. It must define parent links and the expansion rule if parent–child retrieval is used. The **index-update contract** must state when a source update requires re-chunking, re-embedding, deletion, compaction, full rebuild, and publication of a new corpus version. It should prevent a representation generated from one canonical text revision from being presented as evidence for another.

Finally, the **evaluation contract** should measure the declared retrieval and generation task under a fixed corpus revision. It should report unit count, length distribution, duplicate rate, context redundancy, candidate recall, support coverage, latency, and index cost alongside answer quality. Regression examples must include boundary-sensitive cases: a condition following a theorem, a heading that scopes a paragraph, a table whose headers alter interpretation, and code whose signature or import is required to read its body.

10 Summary

Chunking determines the retrieval units that an index can expose. Fixed token windows offer simple and reproducible behavior; sentence, paragraph, structure-aware, and semantic policies seek stronger local coherence at different operational costs. Chunk size and overlap balance specificity, context preservation, index cardinality, and redundant retrieval. Tables, code, and long documents require their structural relations to survive segmentation rather than being flattened into arbitrary text.

Reliable RAG systems treat a chunk as a versioned, attributable source span with metadata and, when useful, a parent relation. A segmentation change is an index change: it may require new representations, new identifiers, and new evaluation. Later chapters can compare sparse and hybrid retrieval, reranking, query transformation, and RAG evaluation, but each depends on whether this chapter's policy made the right evidence retrievable in the first place.

References

[1] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.

[2] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *arXiv preprint arXiv:2401.18059*, 2024, doi: 10.48550/arXiv.2401.18059.